

ASSIGNMENT-9.5

2303A52429

BATCH-32

Problem 1: String Utilities Function

Consider the following Python function:

```
def reverse_string(text):  
    return text[::-1]
```

Task:

1. Write documentation in:

o (a) Docstring

o (b) Inline comments

o (c) Google-style documentation

2. Compare the three documentation styles.

3. Recommend the most suitable style for a utility-based string library.

```
Untitled-1.py > ...  
1  def reverse_string(text):  
2      """  
3      This function takes a string as input and returns the reversed version of that string.  
4      :param text: The string to be reversed  
5      :return: The reversed string  
6      """  
7      """  
8      Args:      text (str): The string to be reversed  
9      Returns:   str: The reversed string  
10     Raises:    None  
11     """  
12  
13     return text[::-1] #slicing is used to reverse the string  
14 input_string = input("Enter a string to reverse: ")  
15 reversed_string = reverse_string(input_string)  
16 print("Reversed string:", reversed_string)
```

```

PS C:\Users\Admin\Downloads\AI ASSISTENCE> C:/Users/Admin/AppData/Local/Programs/Python/Python312/python.exe
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
Ctrl click to launch VS Code Native REPL
>>> def reverse_string(text):
...     """
...     This function takes a string as input and returns the reversed version of that string.
...     :param text: The string to be reversed
...     :return: The reversed string
...     """
...     """
...     Args:
...         text (str): The string to be reversed
...     Returns:
...         str: The reversed string
...     Raises
...     Description: This function uses slicing to reverse the input string. The slicing syntax [::-1
...     ] creates a new string that is a reversed version of the original string. The function takes the input string,
...     reverses it, and returns the reversed string.
...     """
...     return text[::-1]
...

```

Inline Comments

- Written inside the function to explain specific lines of code.
- Good for explaining logic, but not suitable for describing the full purpose, inputs, or errors.

Basic Docstrings

- Placed at the top of the function to explain what it does.
- Simple and readable, but may not clearly structure arguments, returns, or exceptions.

Google Style Docstrings

- Structured with clear sections like Args, Returns, and Raises.
- Best for larger or professional projects because it clearly documents inputs and possible errors.

COMPARISION:

The current combination of inline comments and docstrings is suitable for this code, as it provides both a high-level overview and specific details about the function's behavior. Google style is not so better because the code is simple to solve it is used for complex programs

Problem 2: Password Strength Checker

Consider the function:

```

def check_strength(password):
    return len(password) >= 8

```

Task:

1. Document the function using docstring, inline comments, and Google style.

2. Compare documentation styles for security-related code.
3. Recommend the most appropriate style.

```
Untitled-1.py > check_strength
1  def check_strength(password):
2      """
3      This function checks the strength of a password based on its length.
4      If the password is less than 8 characters long, it is considered weak.
5      Otherwise, it is considered strong.
6      Parameters:
7      password (str): The password to be checked.
8      Returns:
9      str: "Weak" if the password is less than 8 characters, "Strong" otherwise.
10     """
11     """
12     Args:
13     password (str): The password to be checked.
14     Returns:
15     str: "Weak" if the password is less than 8 characters, "Strong" otherwise.
16     Raises:
17     None
18     """
19     if not isinstance(password, str):
20         raise ValueError("Password must be a string")
21     if len(password) < 8:
22         return "Weak"
23     else:
24         return "Strong"
25
```

```
Parameters:
    password (str): The password to be checked for strength.
Returns:
    bool: True if the password is strong (at least 8 characters), False otherwise.

True
```

Inline Comments

- Written inside the function to explain specific lines of code.
- Good for explaining logic, but not suitable for describing the full purpose, inputs, or errors.

Basic Docstrings

- Placed at the top of the function to explain what it does.
- Simple and readable, but may not clearly structure arguments, returns, or exceptions.

Google Style Docstrings

- Structured with clear sections like Args, Returns, and Raises.
- Best for larger or professional projects because it clearly documents inputs and possible errors.

Use the **Google-style structured docstring**.

Security-related functions must clearly document input expectations and exceptions. Structured documentation reduces ambiguity, improves maintainability, and supports auditing.

Problem 3: Math Utilities Module

Task:

1. Create a module `math_utils.py` with functions:

o `square(n)`

o `cube(n)`

o `factorial(n)`

2. Generate docstrings automatically using AI tools.

3. Export documentation as an HTML file.

```
def square(n):  
    """  
    This function takes a number as input and returns its square.  
    """  
    return n * n  
  
def cube(n):  
    """  
    This function takes a number as input and returns its cube.  
    """  
    return n * n * n  
  
def factorial(n):  
    """  
    This function takes a number as input and returns its factorial.  
    """  
    if n < 0:  
        return "factorial is not defined for negative numbers"  
    elif n == 0 or n == 1:  
        return 1  
    else:  
        result = 1  
        for i in range(2, n + 1):  
            result *= i  
        return result
```

```

PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc math_utils
Help on module math_utils:

NAME
    math_utils

FUNCTIONS
    cube(n)
        This function takes a number as input and returns its cube.

    factorial(n)
        This function takes a number as input and returns its factorial.

    square(n)
        This function takes a number as input and returns its square.

FILE
    c:\users\admin\downloads\ai assistance\math_utils.py

PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -w math_utils
wrote math_utils.html
❖ PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -p 1234
Server ready at http://localhost:1234/
Server commands: [b]rowser, [q]uit
server> b
server>

```

Python 3.12.5 [tags/v3.12.5:f3bc82, MSC v.1940 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords

Index of Modules

Built-in Modules

_abc	_imr	_statistics	builtins
_ast	_io	_string	cmath
_bisect	_json	_struct	errno
_blake2	_locale	_svntable	faulthandler
_codecs	_lspref	_thread	gc
_codecs_cn	_md5	_tokenize	itertools
_codecs_hk	_multibytecodec	_tracemalloc	marshal
_codecs_iso2022	_opcode	_typing	math
_codecs_jp	_operator	_warnings	mmap
_codecs_kr	_pickle	_weakref	msvcrt
_codecs_tw	_random	_winapi	nt
_collections	_sha1	_xxinterchannels	os
_contextvars	_sha2	_xxsubinterpreters	sys
_csv	_sha3	array	time
_datetime	_signal	atexit	xximport
_functools	_site	audiopg	xxsubtypes
_heapq	_stat	binascii	zlib

C:\Users\Admin\Downloads\AI ASSISTENCE

20th	aiac	feb_18	lab_ass_8
24 feb	aiac_feb_17	jan_13th	labexam_2
Untitled-1	calculatorexample	lab-7	labexamaiac
Untitled-2	even numbers	lab_8	math_utils

C:\Users\Admin\AppData\Local\Programs\Python\Python312\python312.zip

Python 3.12.5 [tags/v3.12.5:ff3bc82, MSC v.1940 64 bit (AMD64)]
Windows-11

math

This module provides access to the mathematical functions defined by the C standard.

Functions

acos(x, /)
Return the arc cosine (measured in radians) of x.

The result is between 0 and pi.

acosh(x, /)
Return the inverse hyperbolic cosine of x.

asin(x, /)
Return the arc sine (measured in radians) of x.

The result is between -pi/2 and pi/2.

asinh(x, /)
Return the inverse hyperbolic sine of x.

atan(x, /)
Return the arc tangent (measured in radians) of x.

The result is between -pi/2 and pi/2.

atan2(y, x, /)
Return the arc tangent (measured in radians) of y/x.

Unlike [atan\(y/x\)](#), the signs of both x and y are considered.

atanh(x, /)
Return the inverse hyperbolic tangent of x.

cbrt(x, /)
Return the cube root of x.

Problem 4: Attendance Management Module

Task:

1. Create a module `attendance.py` with functions:

- o `mark_present(student)`
- o `mark_absent(student)`
- o `get_attendance(student)`

2. Add proper docstrings.

3. Generate and view documentation in terminal and browse

```

attendance.py > mark_absent
1
2 def mark_present(student):
3     """
4     This function takes a student's name as input and marks them as present.
5     """
6     print(f"{student} is marked as present.")
7 def mark_absent(student):
8     """This function takes a student's name as input and marks them as absent.
9     """
10    print(f"{student} is marked as absent.")
11 def get_attendance(student):
12     """
13     This function takes a student's name as input and returns their attendance status.
14     """
15     return "present"

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\Admin\Downloads\AI ASSISTENCE>
• PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc attendance
Help on module attendance:

NAME
    attendance

FUNCTIONS
    get_attendance(student)
        This function takes a student's name as input and returns their attendance status.

    mark_absent(student)
        This function takes a student's name as input and marks them as absent.

    mark_present(student)
        This function takes a student's name as input and marks them as present.

FILE
    c:\users\admin\downloads\ai assistance\attendance.py

• PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -w attendance
wrote attendance.html
❖ PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -p 2429
Server ready at http://localhost:2429/
Server commands: [b]rowser, [q]uit
server> b
server>

```


Python 3.12.5 [tags/v3.12.5:ff3bc82, MSC v.1940 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords

GetSearch

Index of Modules

Built-in Modules

_abc	_imp	_statistics	builtins
_ast	_io	_string	cmath
_bisect	_json	_struct	errno
_blake2	_locale	_symtable	faulthandler
_codecs	_lsprof	_thread	gc
_codecs_cn	_md5	_tokenize	iicertools
_codecs_hk	_multibytecodec	_tracemalloc	marshal
_codecs_iso2022	_opcode	_typing	math
_codecs_jp	_operator	_warnings	mmap
_codecs_kr	_pickle	_weakref	msvcrt
_codecs_tw	_random	_winapi	nt
_collections	_sha1	_xxinterpchannels	os
_contextvars	_sha2	_xxsubinterpreters	sys
_csv	_sha3	array	time
_datetime	_signal	atexit	winreg
_functools	_sre	audioop	xxsubtype
_heapq	_stat	binascii	zlib

C:\Users\Admin\Downloads\AI ASSISTENCE

20th	aiac	even_numbers	lab_8
24 feb	aiac_feb_17	feb_18	lab_ass_8
Untitled-1	attendance	jan_13th	labexam_2
Untitled-2	calculatorexample	lab-7	labexamaiaac

C:\Users\Admin\AppData\Local\Programs\Python\Python312\python312.zip

Python 3.12.5 [tags/v3.12.5:ff3bc82, MSC v.1940 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords

GetSearch

msilib

Copyright (C) 2005 Martin v. Löwis
Licensed to PSF under a Contributor Agreement.

Package Contents

[schema](#)[sequence](#)[text](#)

Classes

[builtins.object](#)
Binary
CAB
Control
 [RadioButtonGroup](#)
Dialog
Directory
Feature
Table

class **Binary**([builtins.object](#))
 Binary(fname)

 Methods defined here:

 __init__(self, fname)
 Initialize self. See help(type(self)) for accurate signature.

 __repr__(self)
 Return repr(self).

Problem 5: File Handling Function

Consider the function:

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()
```

Task:

1. Write documentation using all three formats.
2. Identify which style best explains exception handling.
3. Justify your recommendation.

```
def read_file(filename:str)->str:
    """
    this function takes a filename as input and returns the contents of the file as a string
    Parameters:
    filename (str): the name of the file to be read
    """
    try:
        with open(filename, 'r') as f:
            return f.read()
    except FileNotFoundError:
        return "file not found"
    except Exception as e:
        return str(e)
print(read_file.__doc__)
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI AS
ISTENCE/attendance.py"

this function takes a filename as input and returns the contents of the file as a string
Parameters:
filename (str): the name of the file to be read
```

EXPLANATION :

Use Google style documentation because it clearly separates arguments, return values, and possible exceptions.

This makes the function easier to understand, maintain, and safely use in real projects.